HopStop: stop mule payments at the first hop

Track: **Real-Time AI for Critical Decisions** | Transaction-time money-mule scoring on IBM LinuxONE with Snap ML and the Telum Integrated Accelerator for AI

Kabir Roy, B.Tech CSE, VIT Bhopal | IBM Z Datathon 4 (17-18 Oct 2026) | Data: IBM Transactions for AML (synthetic, CDLA-Sharing-1.0)

PROBLEM

Mule accounts pass cyber-fraud money onward within minutes, while AML transaction monitoring often runs hours later. Each extra hop spreads the money across more accounts and banks, which makes freezing and recovery harder. HopStop moves the mule check into the payment path: score each outgoing transfer before it settles, and pause it while the money is still close to the victim.

ARCHITECTURE

IBM LinuxONE instance (s390x) - data, features, model and decision stay on one platform

1. Payment stream
IBM AML HI-Small replayed in timestamp order at set rates (Python producer; Kafka if available)

→

2. Feature engine
In-memory per-account state: fan-in/fan-out over 1h/24h, pass-through ratio, time since last credit, new-counterparty flag, amount z-score. Past events only.

→

3. Scoring
LightGBM/XGBoost model imported into IBM Snap ML (AI Toolkit for IBM Z container). tree_format=auto: Integrated Accelerator for AI if present, else CPU.

→

4. Policy layer
Thresholds set by alert budget: allow / step-up / hold. Top-3 reason codes from per-feature contributions.

→

5. Analyst view
Live queue of held payments, reason codes, account hop graph, latency panel (p50/p99, CPU vs accelerator).

Offline path: training and threshold tuning on a time-based split (early days train, later days test). The same feature code runs offline and online to avoid train/serve skew.

WHY IBM Z, CONCRETELY

- **Co-location:** where core payments already run on Z, scoring on the same platform avoids shipping transaction data to an external AI server and removes a network hop from the decision.
- **On-chip inference:** Telum (z16) and Telum II (z17) include an on-chip AI accelerator; Snap ML can map tree ensembles to it. We report measured latency, accelerator vs CPU-only.
- **Data protection:** stored data and logs encrypted with AES-GCM via OpenSSL (CPACF-accelerated on Z). Stretch: run the scorer in an IBM Secure Execution guest if the event environment supports it.

SUCCESS METRICS (TARGETS, TO BE MEASURED)

Metric	Target
p99 end-to-end decision latency	< 10 ms per transfer
Laundering recall at 1% alert rate	Above a velocity-rules baseline
PR-AUC (minority class)	Reported vs published GBT baselines
Chain interruption	% of pattern-file chains flagged by hop 2
Sustained throughput	Measured TPS at the latency target

24-HOUR BUILD PLAN

When	Work
Before event	Study data, design features, define rules baseline (code only if rules allow pre-work)
16 Oct	LinuxONE setup: Python env, Snap ML container, check NNPA with zDNN
H0-4	Load HI-Small, time split, feature engine, rules baseline
H4-9	Train GBT, import to Snap ML, offline metrics
H9-13	Stream replayer + scoring API + policy layer
H13-16	Latency/throughput benchmark, CPU vs accelerator, micro-batching
H16-20	Analyst dashboard, reason codes, chain-interruption metric
H20-24	README, repo cleanup, 3-min video and pitch

RISKS AND MITIGATIONS

Risk	Mitigation
Accelerator not exposed on event VM	Same code on CPU; report honestly; show tree_format=auto path
Accelerator gains need batches	Benchmark batch 1 vs micro-batches; pick per latency budget
Label leakage from graph features	Past-only windows, strict time split
False holds hurt customers	Step-up tier, alert budget, recall at fixed alert rate
Synthetic data realism	Test on LI-Small too; no claims about real-bank numbers

DEMO STORYLINE (3-MINUTE VIDEO)

1. A fan-out laundering chain from the patterns file starts replaying in the stream.
2. Hop 1 transfer is scored on LinuxONE; HopStop returns HOLD with reason codes (e.g., high pass-through ratio, burst of new counterparties).
3. Analyst queue shows the held payment and the account hop graph; downstream hops never receive the funds.
4. Latency panel: p50/p99 for CPU-only vs accelerator; recall vs rules baseline at the same alert rate.

STRETCH GOALS (ONLY AFTER CORE WORKS)

- Score inside an IBM Secure Execution guest, if the event environment supports it.
- Small ONNX model compiled with IBM Z Deep Learning Compiler (zDLC) as a second scorer for comparison.
- Cross-bank view: use the From Bank / To Bank fields to compare one-bank vs ecosystem-level features.
- Tamper-evident log of hold decisions for audit (hash-chained records).

Sources: IBM AML dataset - kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml (Altman et al., "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models", NeurIPS 2023, arXiv:2306.16424). Snap ML on IBM Z - AI Toolkit for IBM Z and LinuxONE (ibmz-accelerated-for-snapml). All performance figures above are targets, not results.